

Computational challenges for the massive production of enriched pre-editions

Simon Gabay¹, Pierre Kuenzli², Raphael Rubino³, Genis Skura⁴, Loris De Biasi²,
Jean-Luc Falcone², Christophe Charpillot², Matteo Romanello¹

¹DH, ²SciCoS, ³FTI, ⁴GSI

Université de Genève

Geneva, Switzerland

{name.surname}@unige.ch

Thibault Clérice, Juliette Janès, Lucence Ing

ALMAAnaCH

INRIA-Paris

Paris, France

{name.surname}@inria.fr

Nicola Carboni

School of Information Sciences

University of Illinois Urbana-Champaign

Champaign, USA

ncarboni@illinois.edu

Abstract—Computational methods in the humanities require large amounts of rich data that exist only in an analogue format (manuscripts, printed books, etc.). However, converting at scale these analogue documents into rich machine-mineable data poses significant problems in terms of computing capacity – an additional second per page resulting potentially in days of processing and a significantly larger footprint. Therefore, we propose a streamlined pipeline limited to the following tasks: (i) layout analysis, (ii) character optical recognition, (iv) pre-editorialization, (v) linguistic normalization, and (vi) named entity recognition. We assess the impact of each task (including pre- and post-processing), testing different strategies to maximise both the speed of execution and the quality of result, relying only on open solutions. This pipeline is used to transform automatically a massive corpus of French theatrical plays (<1,300,000 pages) into enriched pre-editions (< 10,000 books).

Index Terms—Computational humanities, sustainable high-performance computing, modern philology

Computational approaches to text make distance a condition of knowledge [1], but creating this distance is not trivial: it requires large amounts of digital data, while historical documents are analogue. Although engineering advances have enabled significant savings in time compared to manual work, processing collections at scale remains a technical bottleneck that prevents the construction of the necessary massive corpora for research in the years to come. One of the main reasons for this bottleneck is the constantly increasing complexity of a seemingly simple concept, “digitization”, which now goes well beyond simple automatic transcription and includes a series of (very) time-consuming tasks, allowing efficient extraction, precise restructuring, and heavy enrichment of textual information.

Despite these complexities, we think that Fr. Moretti’s prediction, according to whom “in a few years we will have a digital archive with the full texts of (almost) all novels

ever published” [2], is finally coming true, but at the possible cost of a compromise. This new ambition could require a redefinition of efficiency, not understood only as the best quality of the output, but as a trade-off between speed, quality and energy consumption. On the one hand, if an extra second per page has a negligible impact on digital editions or small corpora, it quickly adds up to hundreds of hours of computing time when building large corpora, producing a significantly larger carbon footprint. On the other hand, accelerating the processing cannot come at the expense of quality, as too large a number of errors jeopardizes the new ambition of constructing pre-editions [3], i.e., automatically encoded texts that come as close as possible to philological standards.

This contribution presents the construction of a first massive corpus (<1,300,000 pages) of modern French theatrical plays, spanning from the end of the 16th century to the 20th century. The selected tasks are as follows: (i) layout analysis, (ii) character optical recognition, (iv) pre-editorialization, (v) linguistic normalization, and (vi) named entity recognition. All these tasks are executed following rigorous philological standards to maximise reusability and interoperability, and all models have been retrained with new data and/or new, more efficient architectures. All tools are integrated into an open and streamlined pipeline, optimized for large qualitative corpora.

I. RESEARCH CONTEXT

Recently, the quest for massive [4] and then enriched [5] corpora has become one of the new frontiers of Natural Language Processing (NLP). Although most of these attempts deal with born digital data, we observe similar trends in the humanities [6], but on a smaller scale because research teams interested in long diachrony are inevitably faced with more complex data, i.e. scans of manuscripts or old prints, rather than native machine-readable text. If retrieving the OCRization

Research has been funded by the FNS project N°220833.

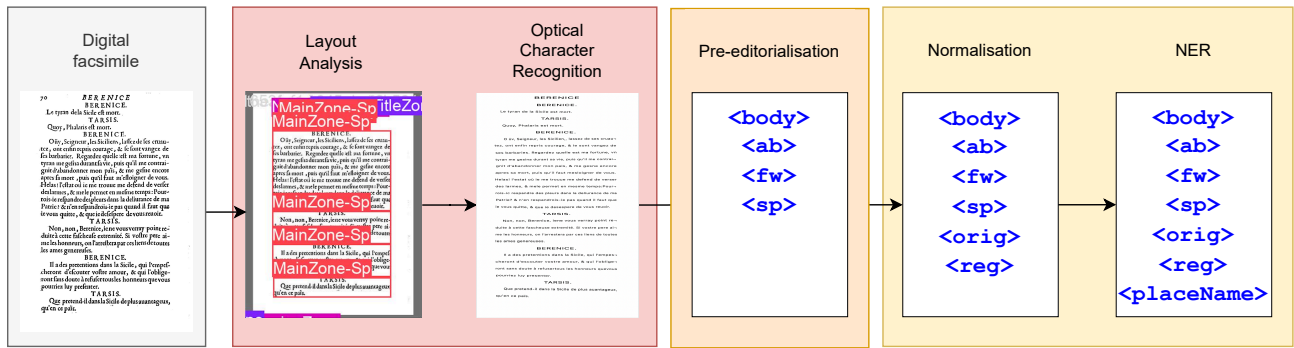


Fig. 1. Production pipeline, with (i) CV tasks in red (text acquisition), (ii) pre-editorialization in orange and (iii) NLP tasks in yellow (text enrichment).

offered by heritage institutions is a practical solution, such an option is not always available, and sometimes not desirable because the quality can be very uneven, particularly for older or complex documents (multiple languages, sophisticated layout, etc.). Therefore, the design of an *ad hoc* pipeline, relying on tools specifically designed for historical data, appears to be a promising solution for the construction and enrichment of large-scale corpora of the best quality [7].

Designing such a pipeline must necessarily take into account collective efforts to standardize computational practices in philology. In recent years, important improvements have been made for Computer Vision (CV) and NLP tasks applied to historical data, with the creation of philological standards for the preparation of data in long diachrony. For French, several guides have been published, whether they are for medieval [8], Renaissance [9] or modern documents [10], offering recommendations in solving recurrent problems and maximizing interchangeability of data.

II. CORPUS

For our experiment, we propose to automatically build, directly from the digital facsimiles, a corpus of approximately 10,000 French theatrical plays. The corpus is made up of more than 1,300,000 pages, retrieved from several digital libraries on top of traditional *Gallica* (Paris)¹: *E-rara* (Switzerland)², *Tolosana* (Toulouse)³, and *Numelyo* (Lyon)⁴. Plays date from the middle of the 16th c. to 1920 – not further because of copyright reasons. Digital facsimiles are not all retrieved via IIIF, some providers limiting (severely) the distribution of images via this framework, but directly from their URL.

The documents present a high level of processing difficulty (fig. 2). The layout is relatively complex for literary

documents, particularly because of the characters' speeches, stage directions, regular complex headings (scene numbers, characters on stage, etc.) or because of a layout on several columns. The printing is not always of good quality, which makes the text difficult to read by machines, and the text presents typographical complexities for the early modern period (ligatures, glyphs that have now disappeared, etc.). The language in which the documents are written varies considerably, ranging from pre-classical French to contemporary French. However, the length in number of pages is limited ($\bar{x} \approx 120$ pages/book⁵), unlike novels, for example, except in the (rare) case of anthologies.

III. MODELS AND TASKS

Our pipeline combines five tasks applied sequentially (fig. 1), with the output of one being the input of the next one, thus presenting a significant risk of propagation of errors that accumulate along the processing chain. There are two main types of tasks: CV and NLP. Between the two, a pre-editorialization step allows the conversion of data into a standard format, XML-TEI [11], with an encoding that is as close as possible to the needs of philologists.

A. Layout analysis

Layout Analysis (LA) detects and classifies the different types of zones on a page (running title, paragraphs, footnotes, etc.). Regarding class types, we use the LADAS framework [12], which derives from the SegmOnto controlled vocabulary [13] (fig. 2). Detection is performed with YOLO [14] v11, an object detection system based on a convolutional neural network (CNN). Several versions of YOLO are available, ranging from nano (2.6M parameters) to extra-large (56.9M parameters).

We use 8,286 pages for training, including 1,157 pages randomly selected within our corpus of plays, with a maximum of one page per document. The input size of the image is 1280 px, and the best mAP50 (0.779) is achieved with the L model (25.3M parameters).

⁵The amount of textual content in a page varies a lot from one book to another, but since the books have not been OCRized yet, it is impossible to provide the number of tokens/book.

TABLE I
CORPUS DESCRIPTION

Docs	11028
Pages/doc.	120.6
Mean res. (w/h)	869/1386
Mean weight	234 ko

¹<https://gallica.bnf.fr>.

²<https://www.e-rara.ch>.

³<https://tolosana.univ-toulouse.fr>.

⁴<https://numelyo.bm-lyon.fr>.

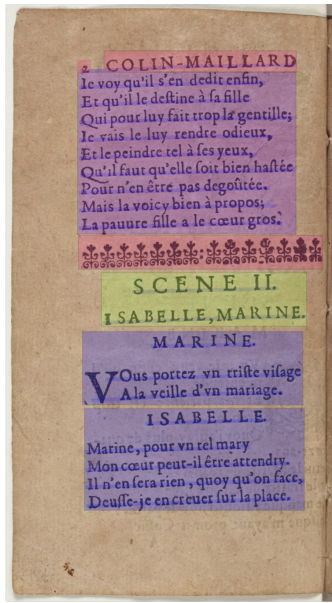


Fig. 2. Automatic classification of the different zones with LADAS.

Fig. 4. Example of pre-editorialization: from the digital facsimile, the data is automatically converted into TEI: MainZone-head (yellow) → <tei:head>, MainZone-sp (purple) → <tei:sp>, etc; or filtered out: GraphicZone:Decoration (pink). Each line is encoded with <tei:lb/>.

Because YOLO and Kraken (§ III-B) use different formats, we use the YALTAi library [15] to convert data from one format to another and to inject the region detection of YOLO in ALTO-XML files. This conversion allows us to use Kraken's line serialization and OCR engine.

B. Optical character recognition

Optical Character Recognition (OCR) converts digital facsimiles into machine-encoded text. We use the Kraken engine, which is based on a hybrid convolutional and recurrent neural network [16]. Its v5 features a new polygon extractor that significantly increases speed and accuracy.

We use 11,899 pages for training, including 5,393 of 19-20th c. French prints and 3,808 of 16-18th c. French prints. It is an enhanced version of the CATMuS *print* model [17], trained on data following the CATMuS guidelines [18]. As explained in [17], this model comes in three versions. The “tiny” and the “small” architectures are those found for different models of Tesseract [19], while the “large” one is the default configuration of Kraken. The difference between the tiny and the small version only is the size of the image provided in input (36 vs 48px), while the large architecture increases dramatically the size of the image (120px) and the number of hidden layers, adding, among other things, several group normalization and dropout layers.

Kraken provides the output in ALTO-XML format, which includes the coordinates of each zone, line, and glyph. The last information being useless for us and extremely verbose (5 lines/glyph), we remove this information from the file to reduce the size of the file (-85%).

```
<pb facs="bpt6k1040272f_page_00022.xml"/>
<fw>2</fw>
<fw>COLIN-MAILLARD</fw>
<lb/>Je voy qu'il s'en dedit enfin,
<lb/>Et qu'il le destine à sa fille
<lb/>Qui pour luy fait trop la gentille;
<lb/>Je vais le luy rendre odieux,
<lb/>Et le peindre tel à ses yeux,
<lb/>Qu'il faut qu'elle soit bien hastée
<lb/>Pour n'en être pas degoutée.
<lb/>Mais la voicy bien à propos;
<lb/>La pauvre fille a le cœur gros.
</sp>
<head>SCENE II.
<lb/>ISABELLE, MARINE</head>
<sp>MARINE
<lb/>VOus porterez vn triste visage
<lb/>A la veille d'vn mariage
</sp>
<sp>ISABELLE
<lb/>Marsine, pour vn tel mary
<lb/>Mon cœur peut-il être attendry.
<lb/>Il n'en sera rien, quoy qu'on face,
<lb/>Deusse-je encreuer sur la place.
</sp>
```

Fig. 3. XML-TEI encoding of the page based on the OCR and the LA.

C. Pre-editorialization

Pre-editorialization converts the output of both the LA and the OCR tasks into machine-actionable data meeting the ecdotic and computational needs of philologists (fig. 4). It provides a pivotal digital state of the text, understood as a common denominator for a maximum number of projects, anticipating a series of time-consuming manipulations, and which it is possible to continue to enrich without having to remove anything.

In practice, a series of XML-ALTO files (or an equivalent format) is converted into an XML-TEI compliant encoding, in which the different zones of the page and the text they contain are organized with the most appropriate XML tessellation for philology. Contrary to previous propositions [20], a conversion of all the XML-ALTO data in the <tei:sourceDoc> of the XML-TEI file has not been considered necessary, because the link to the raw LA/OCR output is maintained by adding the name of the ALTO source file as a value of @facs on <pb> (i.e. “page beginning”).

D. Linguistic normalization

Linguistic Normalization (LN) is a task derived from machine translation, with the particularity that the target is not another language, but another state of the source language (here 17th French to contemporary French). The objective is to obtain several versions of the text that are kept as philological strata (fig. 5), each of them being available independently or simultaneously for research purposes [21]. For our corpus, we retain two strata: the graphematic transcription (the OCR output) and an orthographic normalization (i.e. a text following the rules of contemporary French).

Because the parallel corpus used to train a first model [22] mainly focuses on 17th c. texts, we have added additional

```

<choice>
  <orig><lb/>Pour moy ie vous auouë que<lb/>
  i'en suis tout scandalisé.</orig>
  <reg><lb/>Pour moi je vous avoue que<lb/>
  j'en suis tout scandalisé.</reg>
</choice>

```

Fig. 5. Example of multiple strata encoding.

data from prints of the 16th, 18th, 19th and the beginning of the 20th century. This addition has two objectives: the first is to cover a greater number of states of language (pre-classical French, classical French, contemporary French), the second is to teach the model to modulate its intervention according to needs (the most recent states of language requiring fewer changes than the oldest ones).

The new parallel corpus used for training is made of 32,802 lines. The SMT-based approach has been abandoned for neural-based solutions. One of these solutions, Marian NMT [23], is without a pre-trained model and with a limited amount of parameters (37M). Two others use Transformer-based pretrained language models, m2m100 [24] and mt5 [25], and have a significantly higher number of parameters (respectively 484M and 300M), which should affect processing speed. We also use a transformer-based model with Fairseq [26] using the M architecture described in [22].

E. Named Entity recognition

Named-Entity recognition (NER) is an information extraction task that locates and classifies the named entities mentioned in a text. A dataset has already been prepared to train a model [27]: it follows the *Quaero* guidelines [28], covers a variety of genres (theatre, novels, poetry, etc.) and centuries (mainly 16th-18th c.) and is made of ca. 5,000,000 tokens.

We train three NER models with spaCy [29]. The first one has a CNN architecture with an internal tok2vec model. The two others use a transformer architecture and are based on two BERT models: one is the French pretrained language model CamemBERT [30], and the other one is a version

of CamemBERT fine-tuned on early modern French data: d'AlemBERT [31].

IV. FINAL IMPLEMENTATION AND ESTIMATION

In the following subsections, each task is independently assessed. Different solutions are tested to evaluate the speed/quality ratio and determine eventually the composition of the optimal pipeline. Because the pipeline includes pre- and post-processing phases for each task, it has been decided that the speed measures would include them on top of the sole prediction: the various format conversions and data cleaning tend indeed to be extremely time-consuming, and their exclusion from the calculation would mislead our final choices. With regard to hardware, the alternative between GPUs and CPUs for intensive processes has to be carefully evaluated. GPUs can indeed be much faster, but are rarely accessible in large numbers and are particularly power-hungry. CPUs, on the other hand, are normally accessible in large numbers and therefore easily parallelizable on a large scale, but potentially slower.

The execution time on GPUs and CPUs is tested, when relevant, to identify the best type of processing unit for each task. We used an AMD EPYC 7742 64-Core Processor and a NVIDIA RTX 3090 GPU.

A. Layout analysis and optical character recognition

The OCR code is instrumented to extract the prediction time of the model and the complete inference time, including the pre- and post-processing phases, to properly assess how the time consumption is distributed. The LA code is less easy to instrument: model times on GPU are determined by monitoring GPU usage, while model times on CPU are left blank. Regarding model sizes for OCR and LA, most of the processing time is taken by the pre- and post-processing phases, and the size of the model has a negligible impact on the computing time.

To perform layout analysis and OCR, we decide to use only CPU computing. Indeed, as shown in table II, execution time

TABLE II

BREAKDOWN OF THE RESULTS FOR EACH TASK, WITH GPU VS CPU, AND MODEL INFERENCE TIME VS TOTAL INFERENCE TIME (INCLUDING PRE- AND POST-PROCESSING).

LAYOUT ANALYSIS						
Version	Parameters	GPU		CPU		map50
		Model	Total	Model	Total	
Nano	2.6M	34 s/page	54.8 s/page	NA	58.72 s/page	0.626
Small	9.4M	35 s/page	55.22 s/page	NA	60.4 s/page	0.713
Medium	20.1M	38 s/page	57.29 s/page	NA	63.78 s/page	0.762
Large	25.3M	34 s/page	53.7 s/page	NA	62.35 s/page	0.779
X-large	56.9M	38 s/page	58.9 s/page	NA	65.56 s/page	0.782

OPTICAL CHARACTER RECOGNITION						
Version	Parameters	GPU		CPU		Accuracy
		Model	Total	Model	Total	
Tiny	0.28M	5.33 s/page	52.36 s/page	5.55 s/page	57.52 s/page	98.01%
Small	0.42M	5.58 s/page	53.06 s/page	5.49 s/page	59.60 s/page	98.27%
Large	5.7M	5.58 s/page	53.81 s/page	13.53 s/page	67.98 s/page	98.56%

gains obtained by using a GPU are marginal and they are harder to parallelise.

Books are stored in a shared filesystem, accessible by all computing nodes of the cluster. Each book is stored in a separate folder, and each page is an image file. We use the RTK task management library to apply LA and OCR on batch of books in parallel while keeping disk space usage low [32]. RTK create parallel tasks within a Python framework which themselves apply YALTAi and Kraken on several pages.

RTK works in a shared memory model, that means it can parallelize tasks on a single computing node. To take advantage of the distributed memory architecture of the cluster (the multiple compute nodes), we have to create several parallel jobs, which will themselves run RTK on multiple nodes. We choose to set the level of parallelism of RTK to 10, i.e. a number for which we have not observed any concurrency issues during the execution. That means that each RTK job will run 10 YALTAi and Kraken tasks.

It is then possible to adjust the number of jobs to change the global level of parallelism of this step of the pipeline. To do this, we use a functionality called *job array*⁶, which allows to create as many jobs as necessary, depending on the number of desired batches.

To estimate the time taken to process the full corpus, we setup 5 batches of 10 books and process them in parallel with a job array, each batch being processed by 10 CPU cores. Then, we look at the time taken to process the first batch, which is made of 1,766 pages. This allow to estimate the time taken for one page on one CPU core, and estimate the resources needed to process the full corpus.

With YOLO v11, model LaDaS *large*: 4,313 s / 1,766 pages. Which means it takes 2.442 s/page on 10 cores, or in average 24.42 s/page on one core. Which means it would take

$$4,313s / 1,766pages * 10 * 1,300,000pages \approx 31,749,150s \approx 8,819h$$

to perform this step of the pipeline on the entire corpus on one core (approximately one year)

With Kraken v5, model CATMuS print *large*: 4664 s / 1766 pages. Which means it takes 2.641 s/page on 10 cores, or in average 26.41 s/page on one core. Which means it would take

$$4,664s / 1,766pages * 10 * 1,300,000pages \approx 34,332,955s \approx 9,536h$$

to perform this step of the pipeline on the entire corpus on one core (slightly more than a year).

B. Linguistic Normalization

The most recent architectures offer a gain in quality, but at the cost of a significant increase in processing time. As a result, we are falling back on a Fairseq model, an older solution whose results are satisfactory, but which allows for a very significant gain in processing time. With batch processing (4), we reach 7275 s/1,766 pages, i.e. 0.412 s/page on 10 cores, or an average 4.12 s/page on one core. Which means it would take

⁶https://slurm.schedmd.com/job_array.html

TABLE III
BREAKDOWN OF THE RESULTS FOR LN

Version	Parameters (pretrained)	GPU	CPU	BLEU
Fairseq	- (No)	0.97 s/page	6.71 s/page	91.8
Marian	37M (No)	1.55 s/page	24.28 s/page	93.8
mT5	300M (Yes)	2.65 s/page	19.05 s/page	92.6
M2M-100	484M (Yes)	2.17 s/page	45.65 s/page	94.4

$$7,275s, / 1,766pages * 1,300,000pages \approx 5,355,322s \approx 1,487h$$

to perform this step of the pipeline on the entire corpus on one core.

C. Named Entity recognition

TABLE IV
BREAKDOWN OF THE RESULTS FOR NER

Version	With BERT model	GPU	CPU	Accuracy
SpaCy	No	NA	0.99 s/page	87%
SpaCy	CamemBERT	NA	1.59 s/page	91%
SpaCy	d'Alembert	NA	1.62 s/page	93%

Given the processing time, we do not attempt GPU inference. Since the time difference is marginal between all models, we use the Alembert-based one, which offers the best accuracy. With batch processing (32), we reach 314 s/1,766 pages, i.e. 0.177 s/page on 10 cores, or an average 1.77 s/page on one core. Which means it would take

$$7,275s, / 1,766pages * 1,300,000pages \approx 231144s \approx 64h$$

to perform this step of the pipeline on the entire corpus on one core.

D. Final choices

We have thus, for all tasks, approximately 19,906 CPU hours of computing time. A High-performance computing (HPC) infrastructure is particularly suited for this kind of pipeline because it requires processing data in a massively parallel way. It is a perfectly parallel workload, which does not require communication between tasks, each document being processed independently of others. Because of this, despite the fact that they tend to be slower than GPUs, CPUs are more efficient because they are more numerous in HPC clusters, and are therefore easier to parallelize.

CPUs can be more or less massively parallelized on 100, 500 or 1,000 cores, dividing respectively the total inference time by 100, 500 or 1,000. The institutional HPC cluster at the University of Geneva, Bamboo, is equipped with 5,700 CPU cores and can easily handle such a workload. Because this task is embarrassingly parallel, we can directly divide the total time by the number of cores available to estimate the runtime needed. It would take

- ≈ 199 hours on 100 cores (8.29 days)

- $\approx$ 39.8 hours on 500 cores (1.65 days)
- $\approx$ 19.9 hours on 1000 cores (less than a day)

to create our corpus. It remains possible to speed up or slow down the processing of documents, depending on the philological needs or ecological ambitions of the researchers.

REFERENCES

- [1] F. Moretti, “Conjectures on world literature,” *New Left Review*, vol. 238, pp. 54–68, 2000. [Online]. Available: <https://newleftreview.org/issues/ii1/articles/franco-moretti-conjectures-on-world-literature>
- [2] —, “Style, inc. reflections on seven thousand titles (british novels, 1740–1850),” *Critical Inquiry*, vol. 36, pp. 134–158, 2009. [Online]. Available: <https://doi.org/10.1086/606125>
- [3] A. Pinche, K. Christensen, and S. Gabay, “Between automatic and manual encoding,” in *TEI 2022 conference : Text as data*, Newcastle, United Kingdom, Sep. 2022. [Online]. Available: <https://hal.science/hal-03780302>
- [4] J. Abadji, P. Ortiz Suarez, L. Romary, and B. Sagot, “Towards a Cleaner Document-Oriented Multilingual Crawled Corpus,” in *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, ser. Proceedings of the Thirteenth Language Resources and Evaluation Conference, Marseille, France, Jun. 2022, 12 pages, 6 figures, 2 tables. [Online]. Available: <https://inria.hal.science/hal-03536361>
- [5] O. de Gibert, G. Nail, N. Arefyev, M. Bañón, J. van der Linde, S. Ji, J. Zaragoza-Bernabeu, M. Aulamo, G. Ramírez-Sánchez, A. Kutuzov, S. Pyysalo, S. Oepen, and J. Tiedemann, “A new massive multilingual dataset for high-performance language technologies,” in *LREC-COLING 2024*, Torino, Italia, May 2024, pp. 1116–1128. [Online]. Available: <https://aclanthology.org/2024.lrec-main.100/>
- [6] P.-C. Langlais, J.-B. Camps, N. Baumard, and O. Morin, “From Roland to Conan: First results on the corpus of French literary fictions (1050-1920),” in *DH2022*, Tokyo, Japan, Jul. 2022. [Online]. Available: <https://shs.hal.science/halshs-03715836>
- [7] S. Gabay and T. Clérice, “The birth of French orthography. A computational analysis of French spelling systems in diachrony,” in *CHR2024*, Aarhus, Denmark, Dec. 2024. [Online]. Available: <https://inria.hal.science/hal-04704549>
- [8] A. Pinche, “Guide de transcription pour les manuscrits du Xe au XVe siècle,” Jun. 2022, working paper or preprint. [Online]. Available: <https://hal.science/hal-03697382>
- [9] S. Solfrini, S. Gabay, G. Gross, P.-O. Beaulnes, A. M. Oliveira, and D. Solfaroli Camillocci, “Guide de transcription pour les imprimés français du XVIe siècle en caractères gothiques,” Nov. 2023, working paper or preprint. [Online]. Available: <https://hal.science/hal-04281804>
- [10] S. Gabay, J.-B. Camps, and T. Clérice, “Manuel d’annotation linguistique pour le français moderne (XVIe -XVIIIe siècles),” Apr. 2022, working paper or preprint. [Online]. Available: <https://hal.science/hal-02571190>
- [11] L. Burnard, *What Is the Text Encoding Initiative?* Marseille: OpenEdition Press, 2014. [Online]. Available: <https://doi.org/10.4000/books.oep.426>
- [12] T. Clérice, J. Janes, H. Scheithauer, S. Bénére, F. Cafiero, L. Romary, S. Gabay, and B. Sagot, “Diachronic document dataset for semantic layout analysis,” 2024. [Online]. Available: <https://arxiv.org/abs/2411.10068>
- [13] S. Gabay, A. Pinche, K. Christensen, and J.-B. Camps, “SegmOnto: A Controlled Vocabulary to Describe and Process Digital Facsimiles,” *Journal of Data Mining and Digital Humanities*, Dec. 2024. [Online]. Available: <https://hal.science/hal-04343404>
- [14] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, USA, 2016, pp. 779–788. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.91>
- [15] T. Clérice, “You actually look twice at it (YALTAi): using an object detection approach instead of region segmentation within the kraken engine,” *Journal of Data Mining & Digital Humanities*, Dec. 2023. [Online]. Available: <http://dx.doi.org/10.46298/jdmhdh.9806>
- [16] B. Kiessling, “Kraken - an Universal Text Recognizer for the Humanities,” in *Digital Humanities Conference 2019 - DH2019*, Utrecht, The Netherlands, Jul. 2019. [Online]. Available: <https://doi.org/10.34894/Z9G2EX>
- [17] S. Gabay, T. Clérice, P. Jacsont, E. Leblanc, M. Jeannot-Tirole, S. Solfrini, S. Dolto, F. Goy, C. C. Luján, M. Zaglio, M. Perregaux, J. Janes, B. Sagot, R. Bawden, R. Dent, O. Nédey, and A. Chagué, “Reconnaissance des écritures dans les imprimés,” in *Recueil des communications du 5e colloque Humanistica*, Meknès, Morocco, May 2024. [Online]. Available: <https://hal.science/hal-04557457>
- [18] A. Pinche, T. Clérice, A. Chagué, J.-B. Camps, M. Vlachou-Efstathiou, M. Gille Levenson, O. Brisville-Fertin, F. Boschetti, F. Fischer, M. Gervers, A. Boutreux, A. Manton, S. Gabay, S. Gabay, W. Haverals, M. Kestemont, C. Vandyck, and P. O’Connor, “CATMuS-Medieval: Consistent Approaches to Transcribing Manuscripts,” in *Digital Humanities - DH2024*, Washington DC, United States, Aug. 2024. [Online]. Available: <https://inria.hal.science/hal-04346939>
- [19] A. Kay, “Tesseract: an open-source optical character recognition engine,” *Linux Journal*, vol. 2007, no. 159, p. 2, Jul. 2007. [Online]. Available: <https://dl.acm.org/doi/10.5555/1288165.1288167>
- [20] H. Scheithauer, A. Chagué, S. Gabay, L. Romary, J. Janes, and C. Jahan, “From page to content – which TEI representation for HTR output?” in *Next Gen TEI, 2021*, Weaton (virtual), United States, Oct. 2021. [Online]. Available: <https://hal.science/hal-03380807>
- [21] S. Solfrini, “Normaliser le moyen français : du graphématique au semi-diplomatique,” in *RJC*, Marseille, France, 2025.
- [22] R. Bawden, J. Poinhos, E. Kogktsidou, P. Gambette, B. Sagot, and S. Gabay, “Automatic Normalisation of Early Modern French,” in *LREC 2022 - 13th Language Resources and Evaluation Conference*, Marseille, France, Jun. 2022, pp. 3354–3366. [Online]. Available: <https://inria.hal.science/hal-03540226>
- [23] M. Junczys-Dowmunt, R. Grundkiewicz, T. Dwojak, H. Hoang, K. Heafield, T. Neckermann, F. Seide, U. Germann, A. Fikri Aji, N. Bogoychev, A. F. T. Martins, and A. Birch, “Marian: Fast neural machine translation in C++,” in *Proceedings of ACL 2018, System Demonstrations*, Melbourne, Australia, July 2018, pp. 116–121. [Online]. Available: <http://www.aclweb.org/anthology/P18-4020>
- [24] A. Fan, S. Bhosale, H. Schwenk, Z. Ma, A. El-Kishky, S. Goyal, M. Baines, O. Celebi, G. Wenzek, V. Chaudhary, N. Goyal, T. Birch, V. Liptchinsky, S. Edunov, E. Grave, M. Auli, and A. Joulin, “Beyond english-centric multilingual machine translation,” *J. Mach. Learn. Res.*, vol. 22, no. 1, Jan. 2021. [Online]. Available: <https://dl.acm.org/doi/abs/10.5555/3546258.3546365>
- [25] L. Xue, N. Constant, A. Roberts, M. Kale, R. Al-Rfou, A. Siddhant, A. Barua, and C. Raffel, “mT5: A massively multilingual pre-trained text-to-text transformer,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Jun. 2021, pp. 483–498. [Online]. Available: <https://aclanthology.org/2021.naacl-main.41>
- [26] M. Ott, S. Edunov, A. Baevski, A. Fan, S. Gross, N. Ng, D. Grangier, and M. Auli, “fairseq: A Fast, Extensible Toolkit for Sequence Modeling,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, Minneapolis, Minnesota, USA, 2019, pp. 48–53. [Online]. Available: <https://aclanthology.org/N19-4009>
- [27] P. Ortiz Suarez and S. Gabay, “A Data-driven Approach to Named Entity Recognition for Early Modern French,” in *COLING*, no. 2022.coling-1.327, Gyeongju, South Korea, Oct. 2022, pp. 3722–3730. [Online]. Available: <https://hal.science/hal-04110765>
- [28] S. Rosset, C. Grouin, and P. Zweigenbaum, *Entités nommées structurées: guide d’annotation Quaero*. LIMSI-CNRS, 2011. [Online]. Available: <https://web.archive.org/web/20240413133355/http://www.quaero.org/media/files/bibliographie/quaero-guide-annotation-2011.pdf>
- [29] I. Montani, M. Honnibal, M. Honnibal, A. Boyd, S. V. Landeghem, and H. Peters, “explosion/spacy: v3.7.2: Fixes for apis and requirements,” Oct. 2023. [Online]. Available: <https://doi.org/10.5281/zenodo.10009823>
- [30] L. Martin, B. Muller, P. J. Ortiz Suárez, Y. Dupont, L. Romary, É. de la Clergerie, D. Seddah, and B. Sagot, “CamemBERT: a tasty French language model,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Online, Jul. 2020, pp. 7203–7219. [Online]. Available: <https://aclanthology.org/2020.acl-main.645/>
- [31] S. Gabay, P. Ortiz Suarez, A. Bartz, A. Chagué, R. Bawden, P. Gambette, and B. Sagot, “From FreEM to d’AleMBERT: a large corpus and a language model for early Modern French,” in *LREC*, Marseille, France, Jun. 2022, pp. 3367–3374. [Online]. Available: <https://aclanthology.org/2022.lrec-1.359/>
- [32] T. Clérice, “RTK: Release the krakens,” v. 2024-06-12. [Online]. Available: <https://github.com/PonteInepique/rtk>